MLflow and MLOps: Model Lifecycle Governance

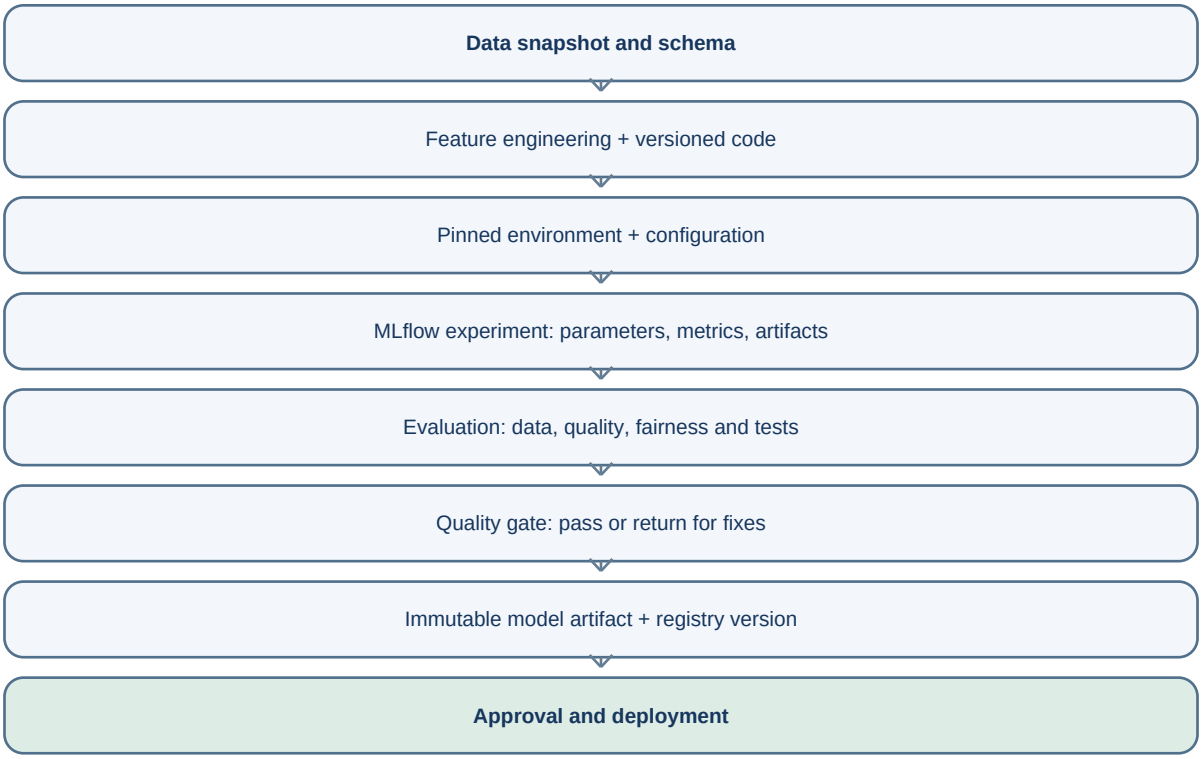
Expanded tutorial discussion material

Learning sequence: lifecycle foundations → experiment tracking and versioning → model governance → quality gates → CI/CD → practical exercise.

1. Connect the ML lifecycle

An industrial ML solution is more than a saved model file. A prediction may depend on the training data, feature transformations, source code, runtime libraries, configuration, model weights, and deployment settings. Connecting these components makes a result traceable and helps teams investigate failures or reproduce a release.

Learning objective: Follow a model from experimentation to a tested, approved, and deployed artifact.



Key idea: if any upstream component changes, record the change and determine whether evaluation or retraining is needed.

2. Versioning and experiment tracking

Versioning answers “exactly what produced this result?” Experiment tracking answers “what happened during this run, and how did it compare with alternatives?” Both are needed for a dependable ML workflow.

- **Traceable versioning:** Record the Git commit, dataset snapshot or version, feature definitions, dependency versions, training configuration, model version, and deployment release identifier.
- **MLflow Tracking:** Log parameters (for example, learning rate or tree count), metrics (F1-score, precision, recall, latency), and artifacts (plots, evaluation reports, confusion matrices, and model files).
- **Dataset and model versions:** Give each training dataset and model a stable identifier. A model version alone does not identify the data or code that produced it.
- **Compare and reproduce runs:** Compare metrics and artifacts across runs. Reproduction also requires the same data, code, environment, configuration, and suitable control of randomness.

Example: Train three Random Forest models with different `n_estimators` values. Log each run, compare validation F1-score and recall, and select a candidate that satisfies the acceptance criteria.

3. Model registry and governance

- **Register and version:** Put qualified candidates in a central model registry so teams can identify, inspect, and refer to a specific model version.
- **Approve and promote:** Define a controlled path such as Candidate → Staging → Production. Promotion should depend on test evidence, review, and approval—not merely on a successful training run.
- **Ownership:** Name the responsible team or individual, the business purpose, supported use cases, and the person or role authorized to approve a release.
- **Model-card evidence:** Record intended use, training-data summary, evaluation methodology and results, limitations, known risks, relevant subgroup performance, and approval history.

Discussion: A model with the highest accuracy is not automatically production-ready. Operational reliability, fairness, security, explainability needs, and business consequences may affect the decision.

4. Quality gates and acceptance criteria

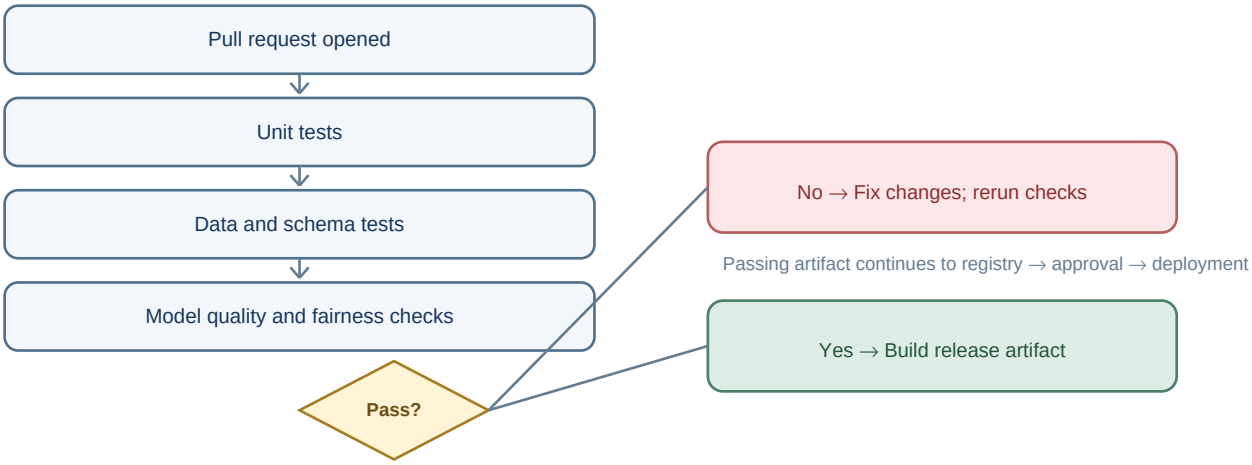
A quality gate is a check that must pass before work can move to the next stage. Thresholds should be agreed before reviewing candidate results, documented, and enforced automatically wherever practical.

Validation area	Example acceptance check
Data schema	Required columns exist; types and allowed values match the contract.
Data quality	Missing values, duplicates, ranges, and unexpected category values stay within limits.
Model quality	Validation F1-score ≥ 0.90 and critical-class recall meets its target.
Bias / fairness	Relevant subgroup metrics are reviewed against approved policy thresholds.
Unit tests	Feature transformations and prediction utilities behave as expected.
Model / API tests	Input validation works; outputs have valid shape, type, and range.
Reproducibility	Code, data, environment, configuration, and run metadata are recorded.

Important: The F1-score threshold above is illustrative. Acceptance criteria depend on the use case, class imbalance, error costs, regulatory obligations, and business risk.

5. CI/CD integration

CI/CD applies software delivery discipline to ML. A pull request should trigger checks against the proposed code and relevant data/model assumptions. A failed check blocks release; a passing check allows the team to create a versioned artifact and continue through governance.



- **Continuous Integration (CI):** Run unit tests, schema and data checks, model evaluation, and any required fairness or security checks when a change is proposed.
- **Continuous Delivery/Deployment (CD):** Package and register the validated model, collect approval where required, and deploy through the defined release process.
- **Immutable artifacts:** Build a uniquely identified artifact only after quality gates pass. Promote the same artifact through test, staging, and production instead of silently rebuilding different binaries at each stage.
- **Audit trail:** Retain the test results, artifact digest or version, approver, release configuration, and deployment reference so the production model can be traced back to its evidence.

6. Practical tutorial exercise

Build a small classification workflow with scikit-learn and MLflow. Keep the exercise focused on the lifecycle rather than on model complexity.

- Train two or three model configurations and log parameters, metrics, and artifacts in MLflow.
- Record the Git commit, dataset version, dependency versions, and configuration.
- Compare runs and select a candidate using agreed validation criteria.
- Add unit tests for preprocessing and prediction, plus data-schema and input-validation tests.
- Define a model-quality threshold and a simple subgroup-performance review where appropriate.
- Simulate a pull request with a failing test; confirm that the release path is blocked.
- Correct the failure, rerun the checks, register the passing model version, document ownership, and record approval.
- Describe how the exact approved artifact would be promoted to production and monitored after deployment.

Expected outcome

Learners should be able to explain how MLflow tracking, dataset and model versioning, quality gates, model governance, and CI/CD fit together. The objective is a release that can be traced, reviewed, reproduced as far as the environment allows, and safely promoted.

Quick review questions

- Which information would you need to reproduce a particular training run?
- Why might a model with better accuracy still fail the release gate?
- What evidence should be available before a model is approved for production?
- Why should the same immutable artifact be promoted across environments?